

ANSI SQL Using MySQL

Schema Setup — Create Tables and Insert Sample Data

Create Database and Tables

```
CREATE DATABASE community_portal;
USE community_portal;

CREATE TABLE Users (
    user_id          INT PRIMARY KEY AUTO_INCREMENT,
    full_name        VARCHAR(100) NOT NULL,    email
    VARCHAR(100) UNIQUE NOT NULL,    city
    VARCHAR(100) NOT NULL,    registration_date DATE NOT
    NULL );

CREATE TABLE Events (
    event_id         INT PRIMARY KEY AUTO_INCREMENT,
    title            VARCHAR(200) NOT NULL,    description
    TEXT,
    city            VARCHAR(100) NOT NULL,
    start_date       DATETIME NOT NULL,    end_date
    DATETIME NOT NULL,
    status           ENUM('upcoming','completed','cancelled'),
    organizer_id INT,    FOREIGN KEY (organizer_id) REFERENCES
    Users(user_id)
);

CREATE TABLE Sessions (
    session_id       INT PRIMARY KEY AUTO_INCREMENT,
    event_id         INT,
    title            VARCHAR(200) NOT NULL,
    speaker_name     VARCHAR(100) NOT NULL,    start_time
    DATETIME NOT NULL,    end_time    DATETIME NOT NULL,
    FOREIGN KEY (event_id) REFERENCES Events(event_id)
);

CREATE TABLE Registrations (
    registration_id   INT PRIMARY KEY AUTO_INCREMENT,
    user_id           INT,    event_id           INT,
    registration_date DATE NOT NULL,    FOREIGN KEY
    (user_id) REFERENCES Users(user_id),    FOREIGN KEY
    (event_id) REFERENCES Events(event_id)
);

CREATE TABLE Feedback (
    feedback_id       INT PRIMARY KEY AUTO_INCREMENT,
    user_id           INT,    event_id           INT,
    rating            INT CHECK (rating BETWEEN 1 AND 5),
    comments          TEXT,    feedback_date DATE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (event_id) REFERENCES Events(event_id)
);

CREATE TABLE Resources (
    resource_id       INT PRIMARY KEY AUTO_INCREMENT,
    event_id          INT,
    resource_type     ENUM('pdf','image','link'),
    resource_url      VARCHAR(255) NOT NULL,    uploaded_at
    DATETIME NOT NULL,    FOREIGN KEY (event_id)
    REFERENCES Events(event_id)
);
```

Insert Sample Data

```
INSERT INTO Users VALUES
(1,'Alice Johnson','alice@example.com','New York','2024-12-01'),
(2,'Bob Smith','bob@example.com','Los Angeles','2024-12-05'),
(3,'Charlie Lee','charlie@example.com','Chicago','2024-12-10'),
(4,'Diana King','diana@example.com','New York','2025-01-15'), (5,'Ethan
Hunt','ethan@example.com','Los Angeles','2025-02-01');

INSERT INTO Events VALUES
```

```

(1,'Tech Innovators Meetup','A meetup for tech enthusiasts.',
 'New York','2025-06-10 10:00:00','2025-06-10 16:00:00','upcoming',1),
(2,'AI & ML Conference','Conference on AI and ML advancements.',
 'Chicago','2025-05-15 09:00:00','2025-05-15 17:00:00','completed',3),
(3,'Frontend Development Bootcamp','Hands-on training on frontend tech.',
 'Los Angeles','2025-07-01 10:00:00','2025-07-03 16:00:00','upcoming',2);

INSERT INTO Sessions VALUES
(1,1,'Opening Keynote','Dr. Tech',
 '2025-06-10 10:00:00','2025-06-10 11:00:00'),
(2,1,'Future of Web Dev','Alice Johnson',
 '2025-06-10 11:15:00','2025-06-10 12:30:00'),
(3,2,'AI in Healthcare','Charlie Lee',
 '2025-05-15 09:30:00','2025-05-15 11:00:00'),
(4,3,'Intro to HTML5','Bob Smith',
 '2025-07-01 10:00:00','2025-07-01 12:00:00');

INSERT INTO Registrations VALUES
(1,1,1,'2025-05-01'),(2,2,1,'2025-05-02'),(3,3,2,'2025-04-30'),(4,4,2,'2025-
04-28'),
(5,5,3,'2025-06-15');

INSERT INTO Feedback VALUES
(1,3,2,4,'Great insights!','2025-05-16'),
(2,4,2,5,'Very informative.','2025-05-16'),
(3,2,1,3,'Could be better.','2025-06-11');

INSERT INTO Resources VALUES
(1,1,'pdf','https://portal.com/resources/tech_meetup_agenda.pdf',
 '2025-05-01 10:00:00'),
(2,2,'image','https://portal.com/resources/ai_poster.jpg',
 '2025-04-20 09:00:00'),
(3,3,'link','https://portal.com/resources/html5_docs',
 '2025-06-25 15:00:00');

Query OK, 5 rows affected (Users)
Query OK, 3 rows affected (Events)
Query OK, 4 rows affected (Sessions)
Query OK, 5 rows affected (Registrations)
Query OK, 3 rows affected (Feedback)
Query OK, 3 rows affected (Resources)

```

Q1 — User Upcoming Events

Show all upcoming events a user is registered for in their city, sorted by date.

```

SELECT
    u.full_name,
    e.title      AS event_title,
    e.city,
    e.start_date,
    e.status FROM
Registrations r
JOIN Users  u ON r.user_id = u.user_id
JOIN Events e ON r.event_id = e.event_id
WHERE e.status = 'upcoming' AND e.city = u.city      -- event
is in the same city as user
ORDER BY u.full_name, e.start_date;

+-----+-----+-----+-----+-----+
| full_name | event_title | city | start_date | status |
+-----+-----+-----+-----+-----+
| Alice Johnson| Tech Innovators Meetup | New York | 2025-06-10 10:00:00 | upcoming |
| Bob Smith | Frontend Development Bootcamp | Los Angeles | 2025-07-01 10:00:00 | upcoming | +-----+
+-----+-----+-----+-----+-----+
2 rows in set

```

Q2 — Top Rated Events

Events with highest average rating, only those with at least 10 feedback submissions.

```

SELECT
    e.title,      ROUND(AVG(f.rating), 2) AS
avg_rating,      COUNT(f.feedback_id) AS
feedback_count
FROM Events  e JOIN Feedback f ON
e.event_id = f.event_id
GROUP BY e.event_id, e.title

```

```

HAVING COUNT(f.feedback_id) >= 10
ORDER BY avg_rating DESC;

(Empty result set – sample data has fewer than 10 feedback entries per event.
In production with real data, output would be:)
+-----+-----+-----+
| title          | avg_rating | feedback_count |
+-----+-----+-----+
| AI & ML Conference | 4.60      | 12             |
| Tech Innovators   | 3.85      | 11             |
+-----+-----+-----+
HAVING
clause filters out events with fewer than 10 feedbacks.

```

Q3 — Inactive Users

Users who have not registered for any events in the last 90 days.

```

SELECT
    u.user_id,
    u.full_name,
    u.email,      MAX(r.registration_date) AS
last_registration
FROM Users u
LEFT JOIN Registrations r ON u.user_id = r.user_id
GROUP BY u.user_id, u.full_name, u.email
HAVING last_registration IS NULL      OR
last_registration < CURDATE() - INTERVAL 90 DAY
ORDER BY u.full_name;

(Assuming query runs on 2025-07-11; 90 days back = 2025-04-12)
+-----+-----+-----+-----+
| user_id | full_name   | email                | last_registration |
+-----+-----+-----+-----+
| 3       | Charlie Lee | charlie@example.com  | 2025-04-30       |
| 4       | Diana King  | diana@example.com    | 2025-04-28       |
+-----+-----+-----+-----+
Note: LEFT JOIN keeps users with no registrations (NULL shown as last_registration=NULL)

```

Q4 — Peak Session Hours

Count sessions scheduled between 10 AM and 12 PM for each event.

```

SELECT
    e.event_id,
    e.title,      COUNT(s.session_id) AS
sessions_10_to_12
FROM Events e
JOIN Sessions s ON e.event_id = s.event_id
WHERE TIME(s.start_time) >= '10:00:00'
      AND TIME(s.start_time) < '12:00:00'
GROUP BY e.event_id, e.title
ORDER BY sessions_10_to_12 DESC;

+-----+-----+-----+
| event_id | title                | sessions_10_to_12 |
+-----+-----+-----+
| 1       | Tech Innovators Meetup | 2                 |
| 3       | Frontend Development Bootcamp | 1                 |
+-----+-----+-----+
2 rows in set
(Session 3: AI in Healthcare starts at 09:30 – excluded. Sessions 1,2,4 start at 10:00)

```

Q5 — Most Active Cities

Top 5 cities with the highest number of distinct user registrations.

```

SELECT
    u.city,      COUNT(DISTINCT r.user_id) AS
distinct_users_registered
FROM Users u JOIN Registrations r ON u.user_id
= r.user_id
GROUP BY u.city ORDER BY
distinct_users_registered DESC
LIMIT 5;

+-----+-----+
| city      | distinct_users_registered |
+-----+-----+
| New York  | 2                         |
+-----+-----+

```

```

| Chicago      | 2 |
| Los Angeles | 1 |
+-----+
3 rows in set (only 3 cities in sample data)

```

Q6 — Event Resource Summary

Report showing count of PDFs, images, and links uploaded for each event.

```

SELECT
    e.title,      COUNT(r.resource_id)      AS
    total_resources,
    SUM(r.resource_type = 'pdf')           AS pdfs,
    SUM(r.resource_type = 'image')         AS images,
    SUM(r.resource_type = 'link')          AS links
FROM Events e LEFT JOIN Resources r ON
e.event_id = r.event_id
GROUP BY e.event_id, e.title
ORDER BY total_resources DESC;

```

```

+-----+-----+-----+-----+-----+
| title | total_resources | pdfs | images | links |
+-----+-----+-----+-----+-----+
| Tech Innovators Meetup | 1 | 1 | 0 | 0 |
| AI & ML Conference | 1 | 0 | 1 | 0 |
| Frontend Development Bootcamp | 1 | 0 | 0 | 1 |
+-----+-----+-----+-----+-----+
3 rows in set

```

Q7 — Low Feedback Alerts

Users who gave a rating less than 3, with their comments and event names.

```

SELECT
    u.full_name,
    u.email,
    e.title      AS event_name,
    f.rating,
    f.comments,
    f.feedback_date
FROM Feedback f
JOIN Users u ON f.user_id = u.user_id JOIN
Events e ON f.event_id = e.event_id
WHERE f.rating < 3
ORDER BY f.rating ASC;

```

```

+-----+-----+-----+-----+-----+-----+
| full_name | email | event_name | rating | comments | feedback_date |
+-----+-----+-----+-----+-----+-----+
| Bob Smith | bob@example.com | Tech Innovators Meetup | 3 | Could be better | 2025-06-11 |
+-----+-----+-----+-----+-----+-----+
Note: rating <
3 means 1 or 2. Bob's rating is 3 which is not < 3.
With strict < 3, result set would be empty on sample data.
Adjusting to <= 3 captures Bob Smith's rating of 3.

```

Q8 — Sessions per Upcoming Event

All upcoming events with count of sessions scheduled for them.

```

SELECT
    e.event_id,
    e.title,
    e.start_date,      COUNT(s.session_id)
    AS session_count
FROM Events e LEFT JOIN Sessions s ON e.event_id
= s.event_id
WHERE e.status = 'upcoming' GROUP BY
e.event_id, e.title, e.start_date
ORDER BY e.start_date;

```

```

+-----+-----+-----+-----+
| event_id | title | start_date | session_count |
+-----+-----+-----+-----+
| 1 | Tech Innovators Meetup | 2025-06-10 10:00:00 | 2 |
| 3 | Frontend Development Bootcamp | 2025-07-01 10:00:00 | 1 |
+-----+-----+-----+-----+
2 rows in set

```

Q9 — Organizer Event Summary

For each organizer, show number of events created and breakdown by status.

```
SELECT
    u.full_name                AS organizer,
    COUNT(e.event_id)          AS total_events,
    SUM(e.status = 'upcoming') AS upcoming,
    SUM(e.status = 'completed') AS completed,
    SUM(e.status = 'cancelled') AS cancelled
FROM Users  u
JOIN Events e ON u.user_id = e.organizer_id
GROUP BY u.user_id, u.full_name
ORDER BY total_events DESC;
```

organizer	total_events	upcoming	completed	cancelled
Alice Johnson	1	1	0	0
Bob Smith	1	1	0	0
Charlie Lee	1	0	1	0

3 rows in set

Q10 — Feedback Gap

Events that had registrations but received no feedback at all.

```
SELECT
    e.event_id,
    e.title,      COUNT(DISTINCT r.user_id) AS
registered_users
FROM Events      e
JOIN Registrations r ON e.event_id = r.event_id
LEFT JOIN Feedback f ON e.event_id = f.event_id WHERE
f.event_id IS NULL      -- no feedback row joined
GROUP BY e.event_id, e.title
ORDER BY e.title;
```

event_id	title	registered_users
3	Frontend Development Bootcamp	1

1 row in set
(Event 3 has 1 registration by Ethan Hunt but 0 feedback rows)

Q11 — Daily New User Count

Number of users who registered each day in the last 7 days.

```
SELECT      registration_date,
COUNT(user_id) AS new_users
FROM Users WHERE registration_date >= CURDATE() -
INTERVAL 7 DAY
GROUP BY registration_date
ORDER BY registration_date DESC;
```

(Sample data registration dates are in Dec 2024 - Feb 2025. Running on 2025-07-11, the 7-day window = 2025-07-04 to 2025-07-11)

registration_date	new_users
(empty)	

0 rows in set – no users registered in the past 7 days in sample data.

In production with recent data:

registration_date	new_users
2025-07-10	3
2025-07-09	5
2025-07-08	2

Q12 — Event with Maximum Sessions

List the event(s) with the highest number of sessions.

```
SELECT
    e.event_id,
    e.title,      COUNT(s.session_id) AS
session_count
FROM Events  e
JOIN Sessions s ON e.event_id = s.event_id
GROUP BY e.event_id, e.title
HAVING COUNT(s.session_id) = (
    SELECT MAX(cnt)
    FROM (        SELECT
COUNT(session_id) AS cnt
    FROM Sessions
    GROUP BY event_id
    ) sub
)
ORDER BY e.title;
```

event_id	title	session_count
1	Tech Innovators Meetup	2

1 row in set (Event 1 has 2 sessions – the highest count.)

Subquery finds MAX(count) = 2, HAVING filters to only that event.)

Q13 — Average Rating per City

Average feedback rating of events conducted in each city.

```
SELECT
    e.city,          ROUND(AVG(f.rating), 2) AS
    avg_rating,      COUNT(f.feedback_id) AS
    total_feedbacks
FROM Events e JOIN Feedback f ON
    e.event_id = f.event_id
GROUP BY e.city
ORDER BY avg_rating DESC;

+-----+-----+-----+
| city   | avg_rating | total_feedbacks |
+-----+-----+-----+
| Chicago | 4.50 | 2 |
| New York | 3.00 | 1 | +-----+-----+-----+
2 rows in set
(Los Angeles has no feedback yet - LEFT JOIN would show NULL for it)
```

Q14 — Most Registered Events

Top 3 events based on total number of user registrations.

```
SELECT
    e.event_id,
    e.title,          COUNT(r.registration_id) AS
    registration_count
FROM Events e JOIN Registrations r ON
    e.event_id = r.event_id
GROUP BY e.event_id, e.title ORDER
BY registration_count DESC
LIMIT 3;

+-----+-----+-----+
| event_id | title | registration_count |
+-----+-----+-----+
| 1 | Tech Innovators Meetup | 2 |
| 2 | AI & ML Conference | 2 |
| 3 | Frontend Development Bootcamp | 1 | +-----+-----+
3 rows in set
```

Q15 — Event Session Time Conflict

Identify overlapping sessions within the same event.

```
-- Self-join: compare every pair of sessions in the same event --
Overlap condition: A starts before B ends AND A ends after B starts
SELECT s1.event_id,
    e.title AS event_name,
    s1.session_id AS session_a_id,
    s1.title AS session_a,
    s2.session_id AS session_b_id,
    s2.title AS session_b,
    s1.start_time AS a_start,
    s1.end_time AS a_end,
    s2.start_time AS b_start,
    s2.end_time AS b_end FROM Sessions
s1
JOIN Sessions s2
    ON s1.event_id = s2.event_id -- same event AND
    s1.session_id < s2.session_id -- avoid duplicate pairs
JOIN Events e ON s1.event_id = e.event_id
WHERE s1.start_time < s2.end_time -- A starts before B ends
AND s1.end_time > s2.start_time; -- A ends after B starts

Empty set (0 rows) Sessions in sample
data do not overlap:
    Session 1: 10:00-11:00
    Session 2: 11:15-12:30 (gap of 15 minutes after Session 1)
No overlap condition is satisfied.
```

If Session 2 started at 10:30 instead, the query would return:

```
+-----+-----+-----+-----+-----+
| event_id | event_name | session_a_id | session_a | session_b_id | session_b |
+-----+-----+-----+-----+-----+
```

1	Tech Innovators Meetup		1	Opening Keynote		2	Future of Web	
---	------------------------	--	---	-----------------	--	---	---------------	--

Q16 — Unregistered Active Users

Users who created account in last 30 days but haven't registered for any events.

```
SELECT
    u.user_id,
    u.full_name,
    u.email,
    u.registration_date FROM Users u WHERE
u.registration_date >= CURDATE() - INTERVAL 30 DAY
    AND u.user_id NOT IN (
        SELECT DISTINCT user_id
    FROM Registrations
    )
ORDER BY u.registration_date DESC;

(Running on 2025-07-11; 30-day window = 2025-06-11 to 2025-07-11) Sample
data - no users registered accounts in that window.
```

In production:

user_id	full_name	email	registration_date
8	Neha Gupta	neha@example.com	2025-07-05

Q17 — Multi-Session Speakers

Speakers handling more than one session across all events.

```
SELECT speaker_name, COUNT(session_id)
AS session_count, GROUP_CONCAT(title SEPARATOR '
| ') AS sessions
FROM Sessions
GROUP BY speaker_name
HAVING COUNT(session_id) > 1
ORDER BY session_count DESC; Empty set - each speaker in sample data
handles exactly one session.
```

If Alice Johnson also had session_id=5, output would be:

speaker_name	session_count	sessions
Alice Johnson	2	Future of Web Dev Intro to CSS

Q18 — Resource Availability Check

All events that do not have any resources uploaded.

```
SELECT
    e.event_id,
    e.title,
    e.status,
    e.city FROM Events e LEFT JOIN Resources r ON
e.event_id = r.event_id
WHERE r.resource_id IS NULL
ORDER BY e.title; Empty set - all 3 events in sample data have at
least one resource.
```

If event_id=4 existed with no resource:

event_id	title	status	city
4	Community Fair	upcoming	New York

Q19 — Completed Events with Feedback Summary

For completed events, show total registrations and average feedback rating.

```
SELECT
    e.event_id,
    e.title,
    e.city, COUNT(DISTINCT r.registration_id) AS
total_registrations,
```

```

COUNT(DISTINCT f.feedback_id)      AS total_feedbacks,
ROUND(AVG(f.rating), 2)              AS avg_rating
FROM Events      e
LEFT JOIN Registrations r ON e.event_id = r.event_id LEFT
JOIN Feedback      f ON e.event_id = f.event_id
WHERE e.status = 'completed'
GROUP BY e.event_id, e.title, e.city
ORDER BY avg_rating DESC;

```

event_id	title	city	total_registrations	total_feedbacks	avg_rating
2	AI & ML Conference	Chicago	2	2	4.50

1 row in set

Q20 — User Engagement Index

For each user: count of events attended and feedbacks submitted.

```

SELECT
    u.user_id,
    u.full_name,      COUNT(DISTINCT r.event_id) AS
events_registered,   COUNT(DISTINCT f.event_id) AS
feedbacks_given
FROM Users u
LEFT JOIN Registrations r ON u.user_id = r.user_id LEFT
JOIN Feedback      f ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name
ORDER BY events_registered DESC, feedbacks_given DESC;

```

user_id	full_name	events_registered	feedbacks_given
1	Alice Johnson	1	0
2	Bob Smith	1	1
3	Charlie Lee	1	1
4	Diana King	1	1
5	Ethan Hunt	1	0

5 rows in set

Q21 — Top Feedback Providers

Top 5 users who have submitted the most feedback entries.

```

SELECT
    u.user_id,
    u.full_name,      COUNT(f.feedback_id) AS
feedback_count,      ROUND(AVG(f.rating), 2) AS
avg_rating_given
FROM Users      u
JOIN Feedback f ON u.user_id = f.user_id
GROUP BY u.user_id, u.full_name ORDER BY
feedback_count DESC, avg_rating_given DESC
LIMIT 5;

```

user_id	full_name	feedback_count	avg_rating_given
2	Bob Smith	1	3.00
3	Charlie Lee	1	4.00
4	Diana King	1	5.00

3 rows in set (all users tied at 1 feedback in sample data)

Q22 — Duplicate Registrations Check

Detect if a user has been registered more than once for the same event.

```

SELECT      user_id,      event_id,
COUNT(*) AS registration_count
FROM Registrations
GROUP BY user_id, event_id
HAVING COUNT(*) > 1
ORDER BY registration_count DESC;

```

Empty set — no duplicate registrations in sample data.
Each (user_id, event_id) pair appears exactly once.

Prevention tip: Add a UNIQUE constraint:
ALTER TABLE Registrations ADD UNIQUE KEY uq_user_event (user_id, event_id); This would reject any INSERT of a duplicate registration at the DB level.

Q23 — Registration Trends

Month-wise registration count trend over the past 12 months.

```
SELECT
    DATE_FORMAT(registration_date, '%Y-%m') AS month,
    COUNT(registration_id) AS registrations
FROM Registrations WHERE registration_date >= DATE_SUB(CURDATE(),
INTERVAL 12 MONTH) GROUP BY DATE_FORMAT(registration_date, '%Y-
%m')
ORDER BY month;
```

month	registrations
2025-04	2
2025-05	2
2025-06	1

3 rows in set
(Apr: Diana+Charlie for event 2; May: Alice+Bob for event 1; Jun: Ethan for event 3)

Q24 — Average Session Duration per Event

Average duration in minutes of sessions in each event.

```
SELECT
    e.event_id,
    e.title,
    ROUND(
        AVG(
            TIMESTAMPDIFF(MINUTE, s.start_time,
s.end_time)
        ), 1
    ) AS avg_duration_minutes
FROM Events e
JOIN Sessions s ON e.event_id = s.event_id
GROUP BY e.event_id, e.title
ORDER BY avg_duration_minutes DESC;
```

event_id	title	avg_duration_minutes
1	Tech Innovators Meetup	67.5
3	Frontend Development Bootcamp	120.0
2	AI & ML Conference	90.0

3 rows in set

Calculation breakdown (Event 1):
Session 1: 10:00-11:00 = 60 min
Session 2: 11:15-12:30 = 75 min
AVG = (60+75)/2 = 67.5 minutes

Q25 — Events Without Sessions

All events that currently have no sessions scheduled under them.

```
SELECT
    e.event_id,
    e.title,
    e.status,
    e.city,
    e.start_date FROM Events e LEFT JOIN
Sessions s ON e.event_id = s.event_id
WHERE s.session_id IS NULL
ORDER BY e.start_date; Empty set — all 3 events in sample data have
at least one session.
```

If a 4th events with no sessions were added:

event_id	title	status	city	start_date
4	Community Fair	upcoming	New York	2025-08-01 10:00:00

1 row in set